GSS ADMIN

1.1 Operational Scope

GSS Admin is the platform governance role responsible for:
• master configuration management
• client governance
• user governance
• verification configuration
• operational supervision
• dashboard monitoring

1.2 Main Screens

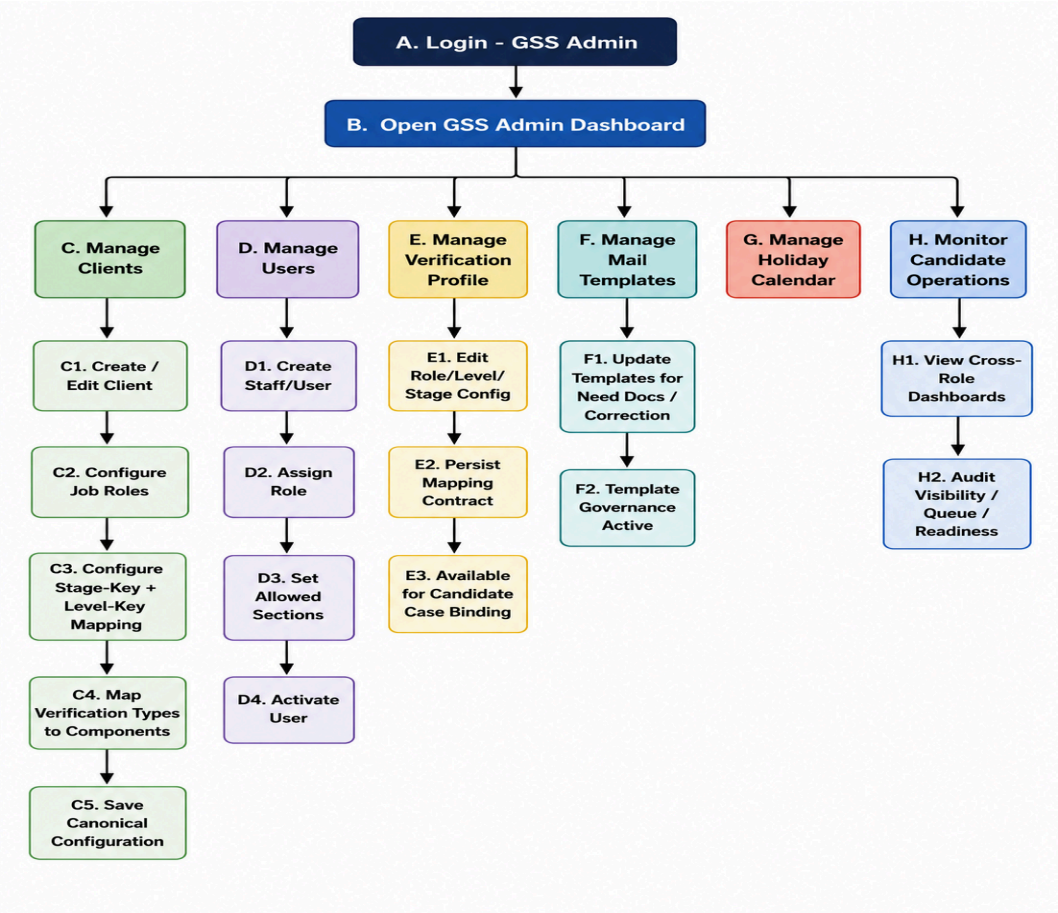
• GSS Admin Dashboard
• Clients List
• Create Client / Edit Client
• Users List
• Create User / Create Staff User
• Candidate List
• Create Candidate Case
• Candidate Bulk Upload
• Verification Profiles List
• Verification Profile Edit
• Mail Templates List
• Mail Template Edit
• Holiday Calendar

1.3 Current Functional Scope

• Dashboard metrics, trends, recent cases, and upcoming holidays are live.
• Client create/update/list/get operations.
• GSS User create/update/list/get operations are functional including:
 - role checks
 - location mapping
 - temporary password setup
 - optional mail send
• Candidate operations are functional for:
 - create
 - list
 - bulk upload
 - report access
• Verification configuration is functional for:
 - job roles
 - stage configuration

- verification type mappings
- verification profiles

• Mail template management is functional.
• Holiday management is functional.
• OTP login flow with role-based routing is functional.



2) Client Admin

2.1 Operational Scope

Client Admin handles client-side onboarding and operational tracking.
Responsibilities include:

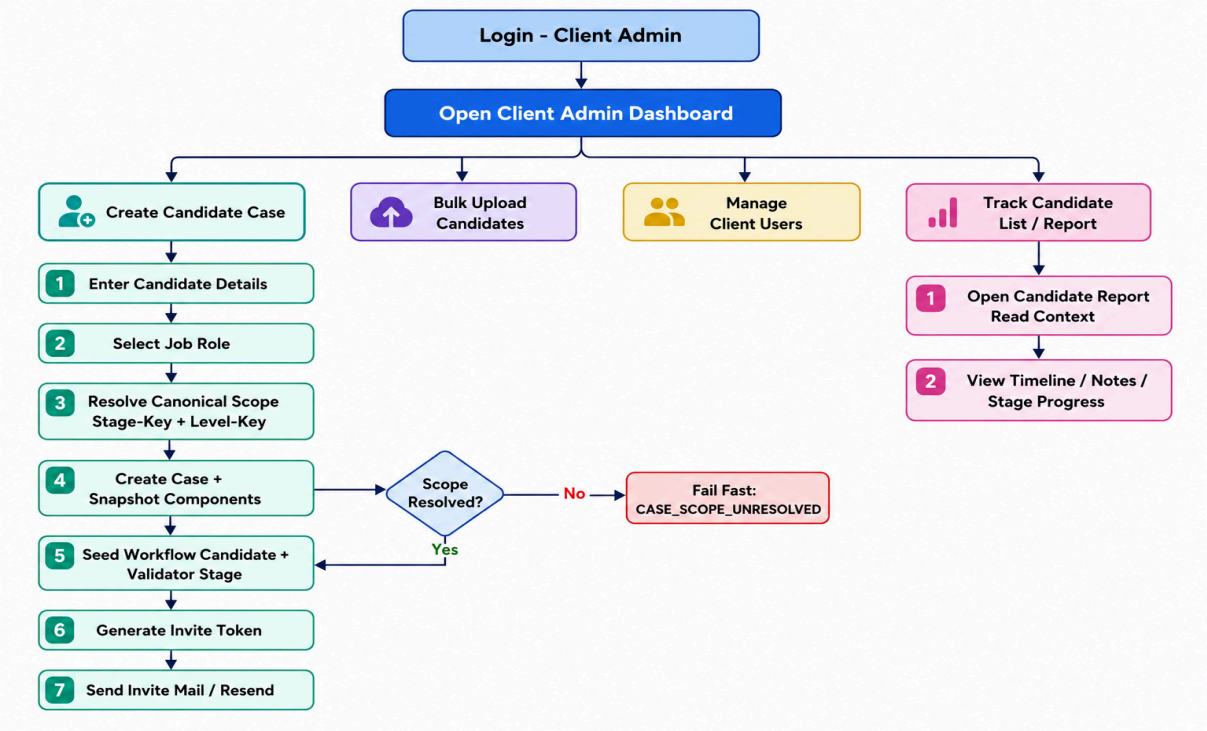
• candidate creation
• candidate invite operations
• candidate tracking
• report visibility
• client-user management

2.2 Main Screens

- Client Admin Dashboard
- Candidate List
- Create Candidate
- Bulk Upload
- Candidate View (Candidate Report)
- Users List
- Create User
- Overall Report

2.3 Current Functional Scope

- Client-scoped dashboard KPIs, trends, status mix, and ageing metrics are live.
- Candidate creation is functional with:
  - input validation
  - duplicate-creation guard
- Invite flow is functional:
  - token generation
  - URL generation
  - resend support
- Client user management is functional.
- Client-scoped case list with search and derived stage labels is functional.
- Shared candidate report rendering is functional in client-admin mode.



3) Validator

3.1 Operational Scope

Validator executes first-stage operational verification.
Responsibilities include:
• queue execution
• component review
• workflow decision actions
• progression toward verifier stage

3.2 Main Screens

• Validator Dashboard
• Validator Candidate List
• Validator Candidate View (Candidate Report)

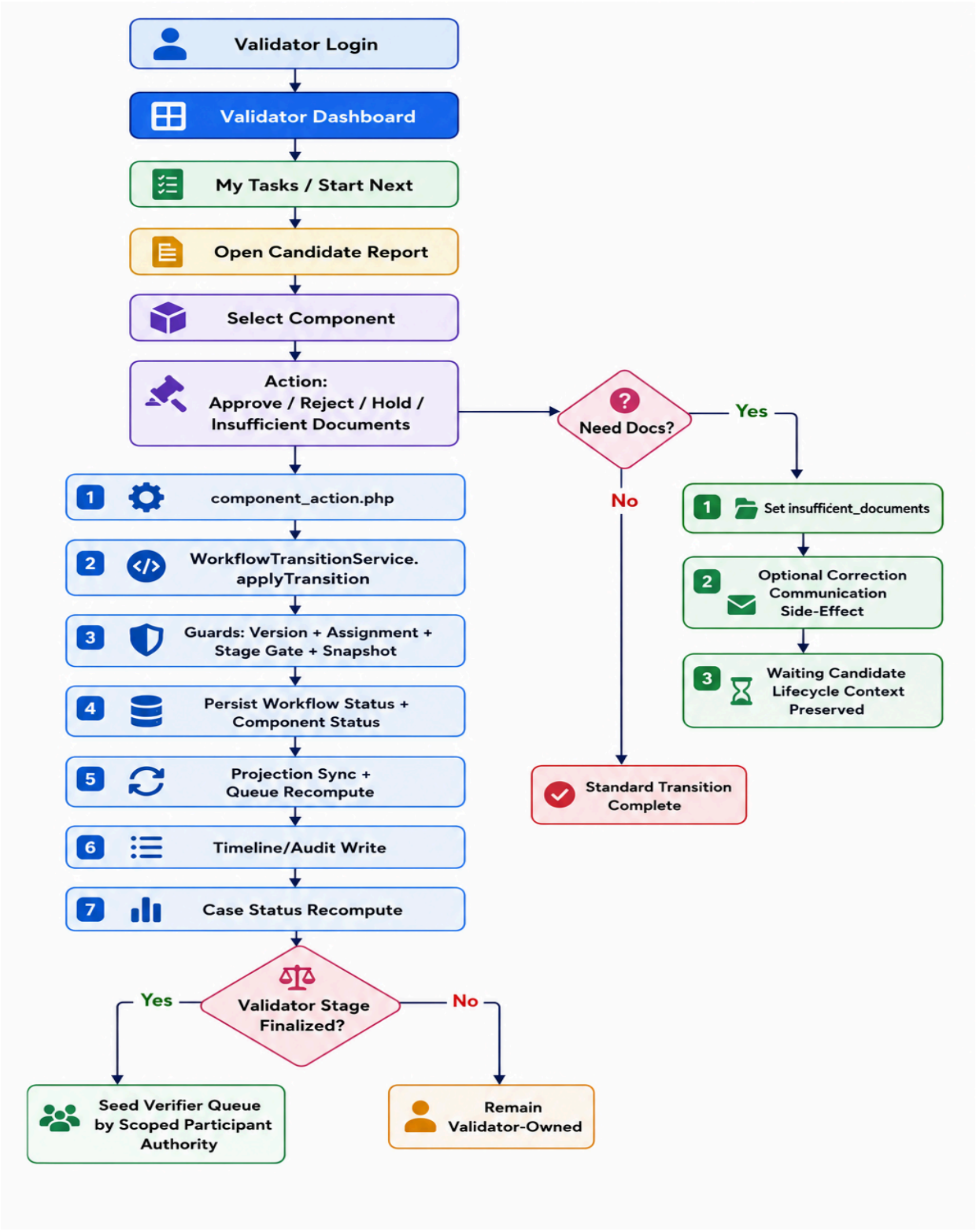
3.3 Current Functional Scope

• Queue dashboard is live with:
- pending
- in-progress
- completed today
- my tasks
• Start Next / claim / complete queue cycle is implemented.
• Candidate list fetch and open flow is implemented.
• Component-level actions are implemented:
- Approve
- Reject
- Hold
- Insufficient Documents
• Reason validation is enforced for:
- Hold,
- Reject
- Insufficient Documents

- All workflow mutations are routed through the centralized workflow transition engine to maintain workflow consistency and audit continuity.
- Action updates automatically synchronize:
  -workflow state
  -timeline logs
  -operational queues
  -transition audit history
- Need Docs (insufficient_documents) follows the same governed workflow transition path.
- Communication and correction mail operations execute as post-transition side effects to preserve workflow integrity.

3.4 Governance

• Projection-driven queue synchronization is enforced.
• Lifecycle progression is derived from canonical workflow summaries.
• Versioned transition guards and idempotency are enforced.
• Downstream activity locking is enforced for operational safety.


Validator Login
Validator Dashboard
My Tasks / Start Next
Open Candidate Report
Select Component
Action:
Approve / Reject / Hold /
Insufficient Documents
Need Docs?
Yes
No
1 Set insufficient_documents
2 Optional Correction Communication Side-Effect
3 Waiting Candidate Lifecycle Context Preserved
Standard Transition Complete
1 component_action.php
2 WorkflowTransitionService. applyTransition
3 Guards: Version + Assignment + Stage Gate + Snapshot
4 Persist Workflow Status + Component Status
5 Projection Sync + Queue Recompute
6 Timeline/Audit Write
7 Case Status Recompute
Validator Stage Finalized?
Yes
No
Seed Verifier Queue by Scoped Participant Authority
Remain Validator-Owned

4) Verifier

4.1 Operational Scope

Verifier executes grouped operational verification after validator stage and progresses cases toward QA.
Responsibilities include:
• grouped verification execution
• queue operations
• component-level decisions
• participation continuity
• QA progression

4.2 Main Screens

• Verifier Dashboard
• Verifier Candidate List
• Verifier Candidate View (Candidate Report)

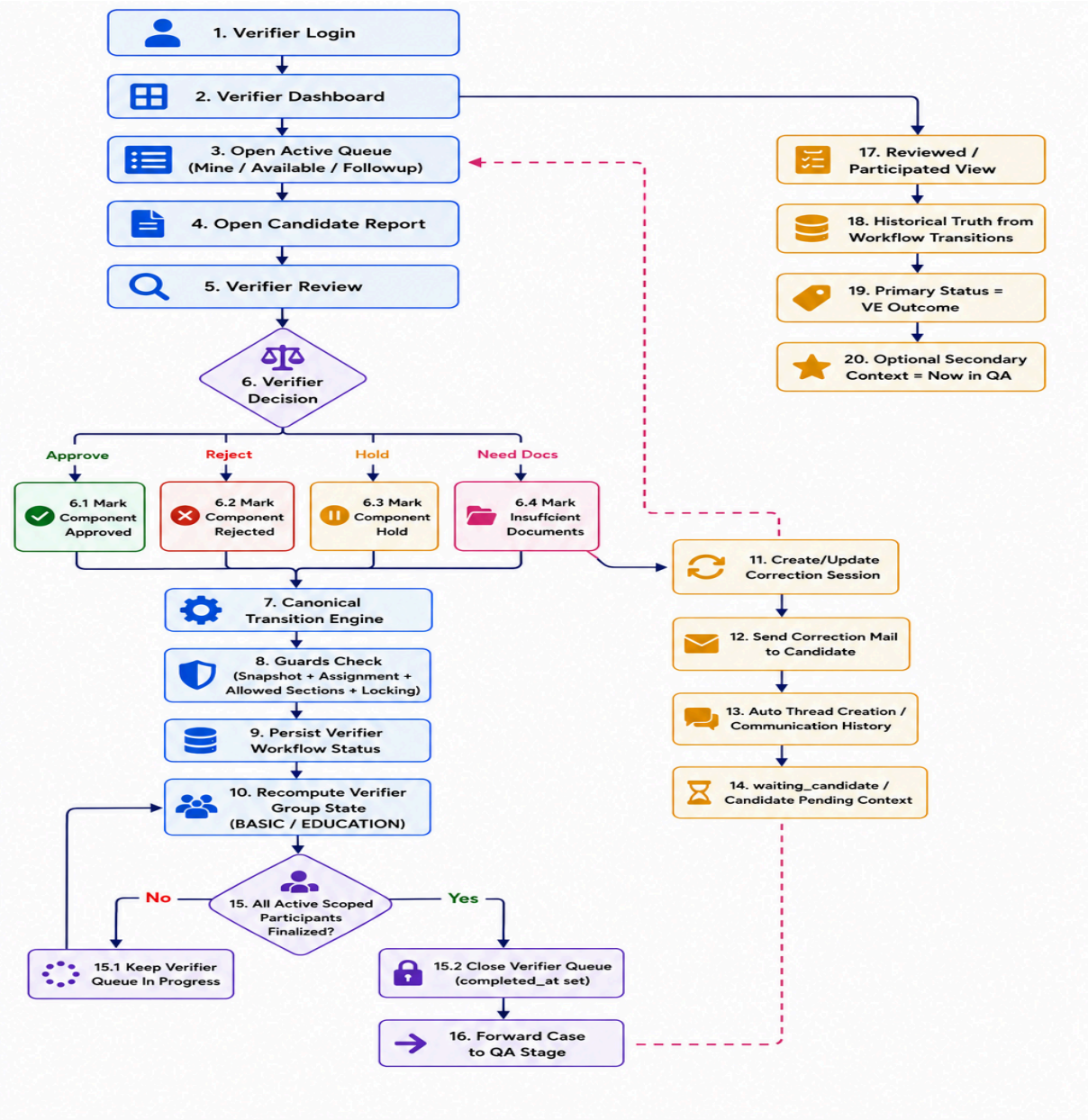
4.3 Current Functional Scope

• Dashboard is live with:
 - queue KPIs
 - assigned module/group handling
• Group-based queue execution is implemented for:
 - BASIC
 - EDUCATION
• Queue operations implemented:
 - list mine
 - list available
 - claim
 - next
 - complete
 - stats
• Component-level actions are implemented:
 - Approve
 - Reject
 - Hold
 - Insufficient Documents
• Reason validation is enforced for:
 - Hold
 - Reject
 - Insufficient Documents
 - rejection overrides
• Active workload visibility is driven through operational queue handling using open and actionable workflow rows.
• Participated/history visibility is maintained through workflow transition audit continuity to preserve historical operational actions
.• Historical verifier continuity is preserved through:
 - VE APPROVED
 - VE REJECTED
 - VE HOLD
 - VE NEED DOCS

• Back-navigation and participated/history continuity are stabilized.

4.4 Verifier Governance

- Verifier authority is scoped only to operationally assigned and workflow-seeded components.
- Queue closure follows deterministic completed_at governance to maintain operational consistency.
- Artificial queue reopening is intentionally avoided to preserve workflow integrity.
- Need Docs and correction workflows follow transition-first governance, with communication activities executed after successful workflow updates.
- Communication thread continuity is preserved throughout the workflow lifecycle.
- Snapshot authority is enforced, allowing workflow mutations only for components officially bound to the candidate case snapshot.



5) QA

5.1 Operational Scope

QA handles final-stage review visibility, readiness validation, and final-stage workflow actions.

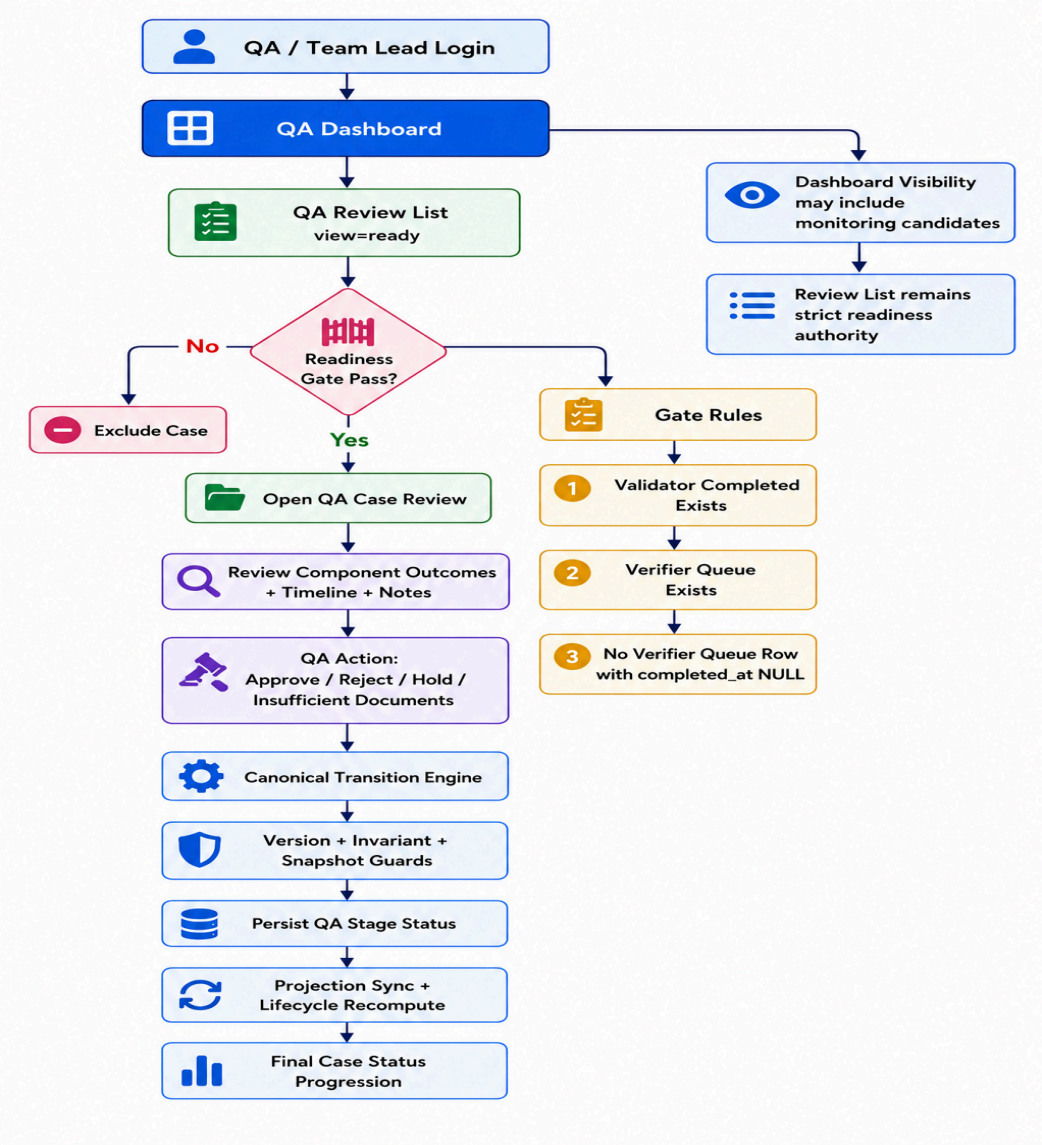
5.2 Main Screens

- QA Dashboard
- QA Review List
- QA Case Review
- QA Reports Tracking

5.3 Current Functional Scope

- QA dashboard is live with workforce and assignment telemetry.
- Case list supports role-based visibility:
  - QA role uses strict ready view
  - Team Lead mode supports broader supervisory filters
- Final-stage actions are implemented through shared workflow action layer:
  - Approve
  - Reject
  - Hold
  - Insufficient Documents
- Reason validation is enforced for:
  - Hold
  -Reject
  - Insufficient Documents
- QA reject finalization logic is implemented.
- QA readiness gate is deterministic:

  verifier queues must be fully resolved with completed_at governance before QA intake.



6) Cross-Role Reality (Current Implementation)

6.1 Centralized Governance

• Role-based report rendering and access are centralized through shared candidate report flow.

• Component actions and workflow transitions are centralized through shared workflow/action services.

• Timeline and audit writes are centralized.

• Operational labels are governed through centralized status governance helpers.

6.2 Operational Implications

• Most role behavior is enforced through shared workflow/action logic.

• Role-specific modules primarily manage:

- queue handling
- dashboard handling
- list visibility

• Communication flows operate as side-effect services.

• Workflow mutation authority remains centralized.

• Advanced automation engines such as SLA/escalation systems are intentionally not introduced in current stabilization scope.

7) Critical Governance Additions

The following stabilization governance improvements are implemented:

• Canonical action routing normalization completed across:

- approve
- reject
- hold
- insufficient_documents

• Need Docs correction loop integrated with:

- waiting_candidate semantics
- resend access continuity

• Auto-thread creation and communication lineage continuity implemented for correction and verification messaging.

• Component-level downstream activity locking introduced for cross-user safety.

• Active workload vs participated/history semantic separation stabilized across:

- dashboard
- list
- report surfaces

• Snapshot membership authority enforced for all workflow mutations.

• Non-snapshot workflow actions are rejected intentionally by design.

• Configuration authority stabilized around:

- stage_key
- level_key

resolution contract for candidate component binding.

- Forensic observability enabled through:
  WF_STATUS_DEBUG_LOGS=1

8) Final Operational Summary

- GSS Admin provides centralized configuration, governance, and operational supervision.
- Client Admin provides onboarding, candidate tracking, invite management, and report visibility.
- Validator provides first-stage operational verification through canonical transition governance.
- Verifier provides grouped operational verification with participation continuity and QA progression.
- QA provides deterministic ready-gate review and final-stage workflow governance.

Overall platform state:

- centralized workflow transition governance
- projection-driven queue synchronization
- snapshot-backed mutation authority
- participated/history continuity
- strict readiness and queue guards
- stabilized operational semantic separation

GSS VATI WORKFLOW